Nikhil Kale
47

Experiment No. 5

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. **Primitive Data Types, Variables, Functions – pure, view**

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int**: unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool**: represents logical values (true or false).
- **address**: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string**: store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure**: cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view**: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. **Inputs and Outputs to Functions**

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. **Visibility, Modifiers and Constructors**

- **Function Visibility** defines who can access a function:
    - public: available both inside and outside the contract.
    - private: only accessible within the same contract.
    - internal: accessible within the contract and its child contracts.
    - external: can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

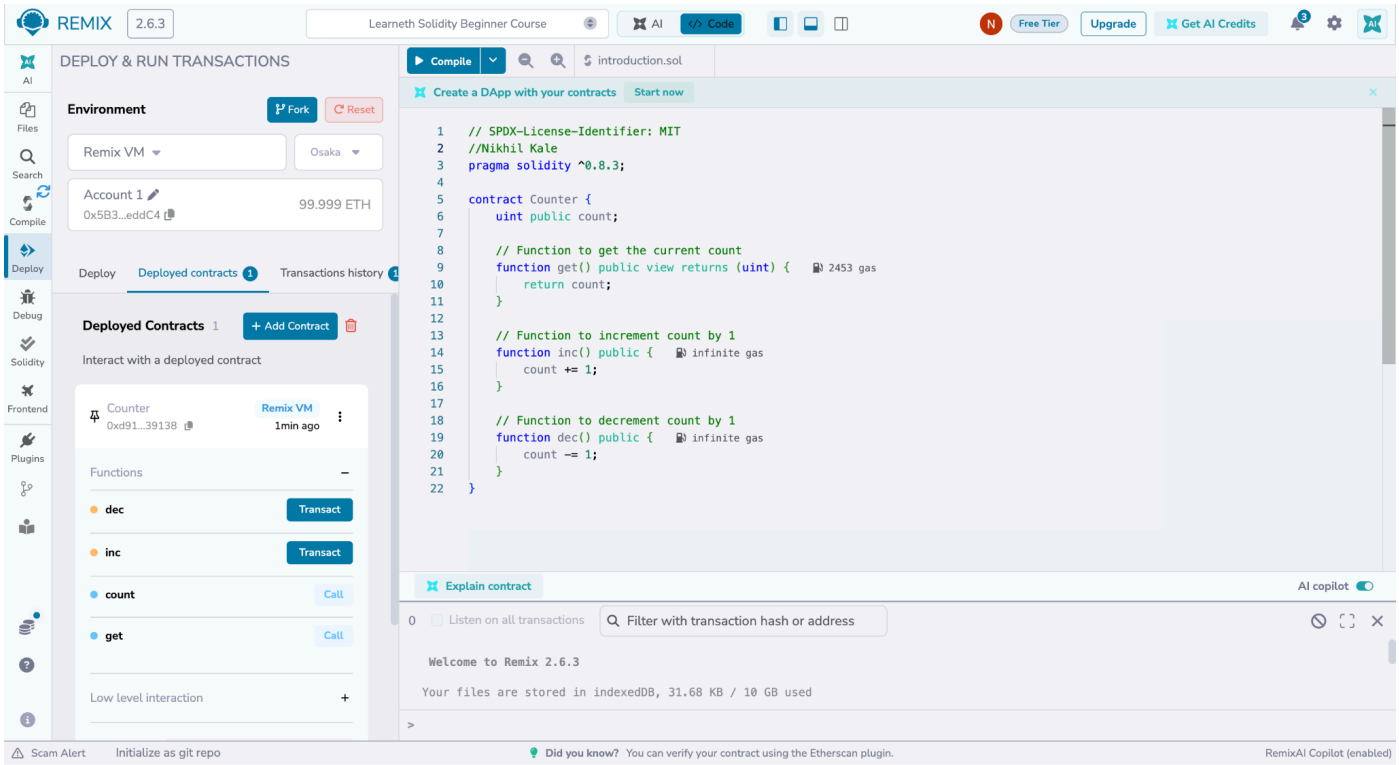
- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

5. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

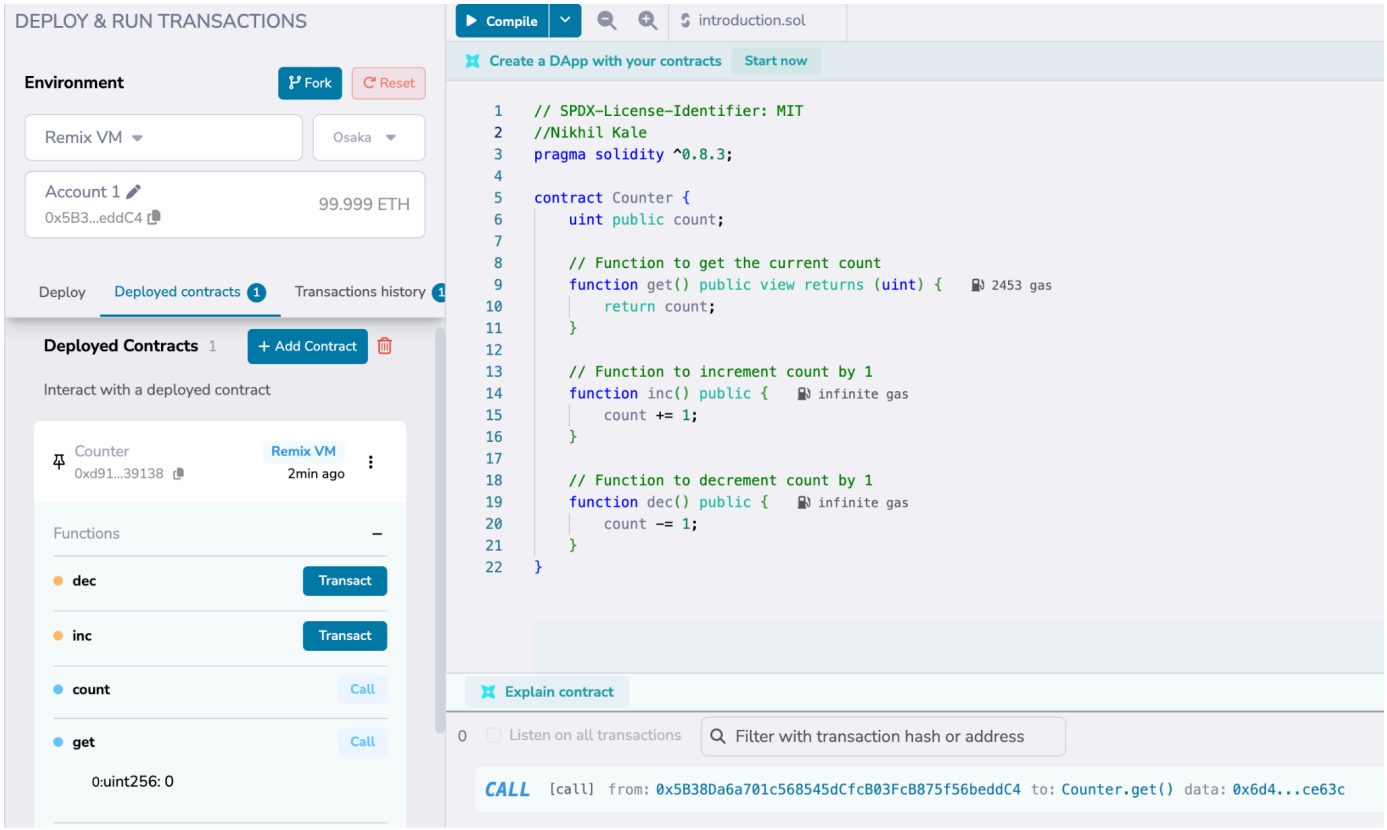
- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

 Implementation:

Tutorial No. 1 - Compile and Deploy Contract



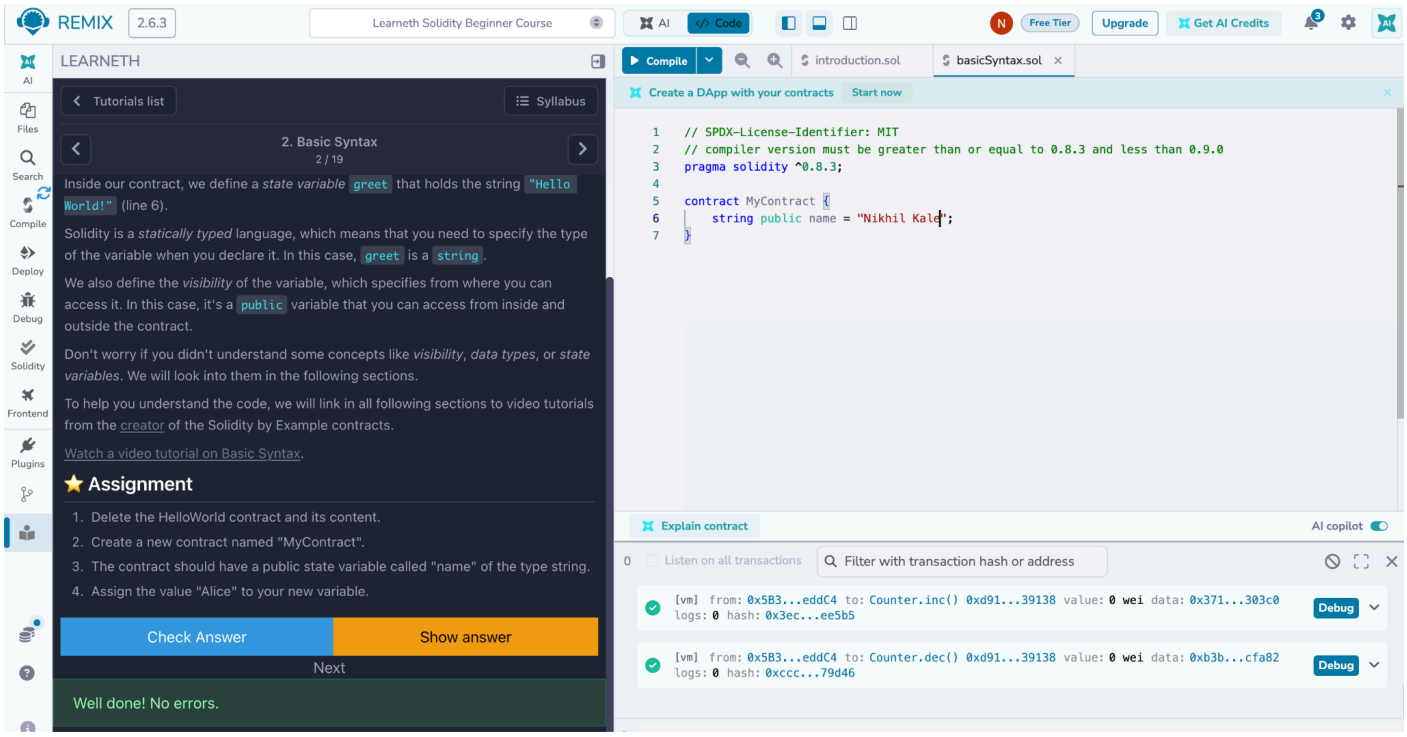
Tutorial No. 1 - get



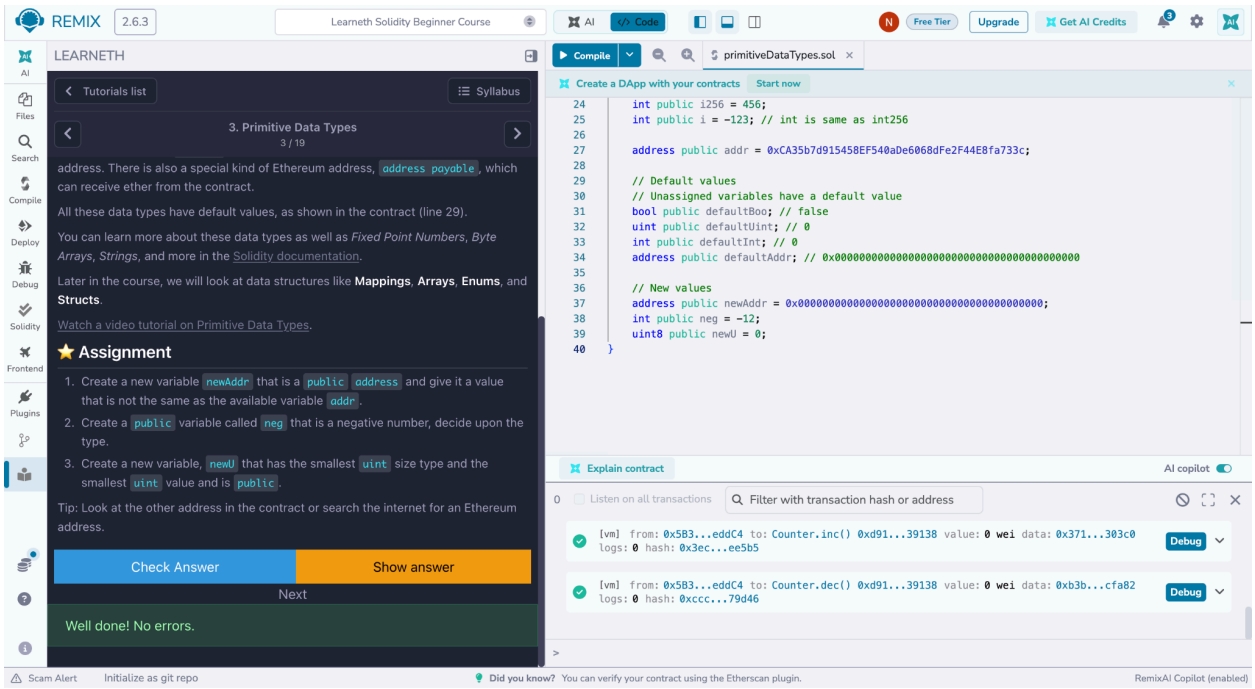
Tutorial No.1 - increment and decrement



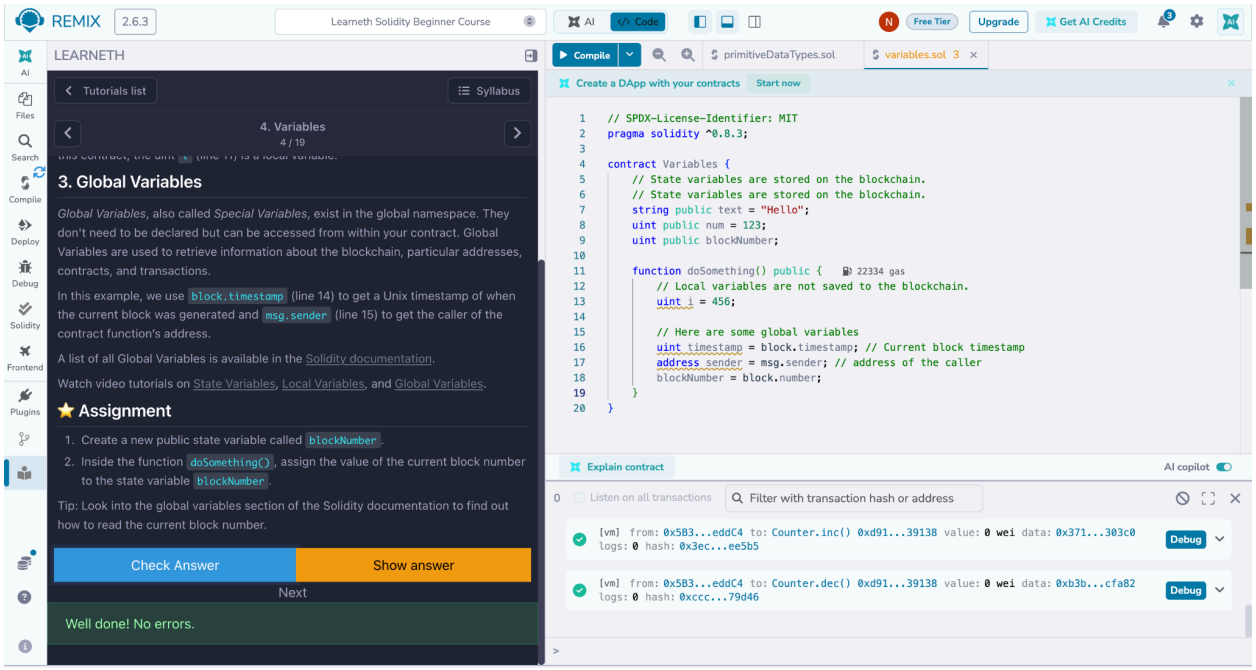
Tutorial No. 2 -



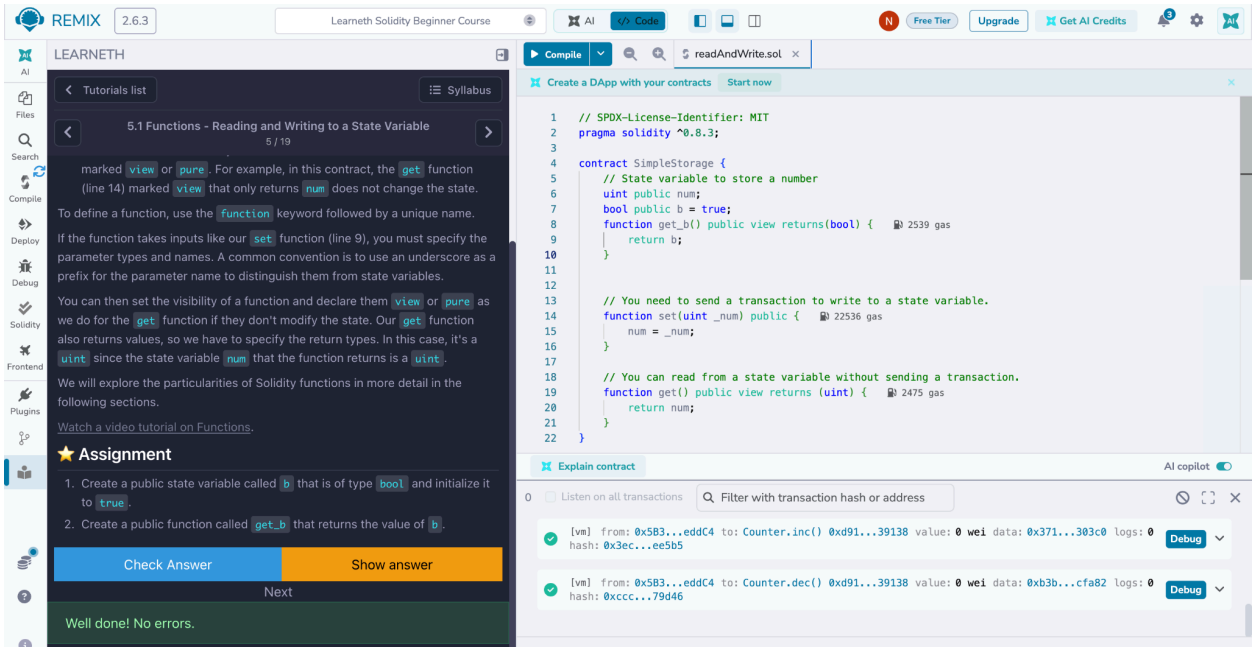
Tutorial 3-



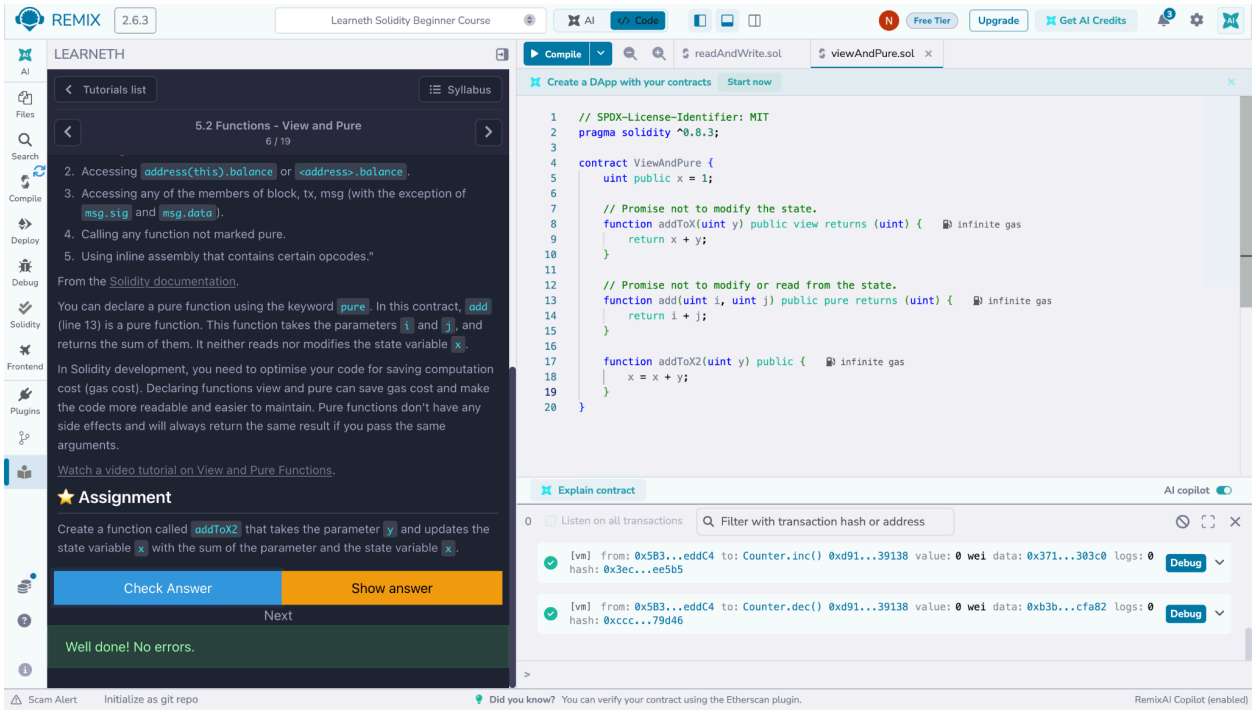
Tutorial 4 -



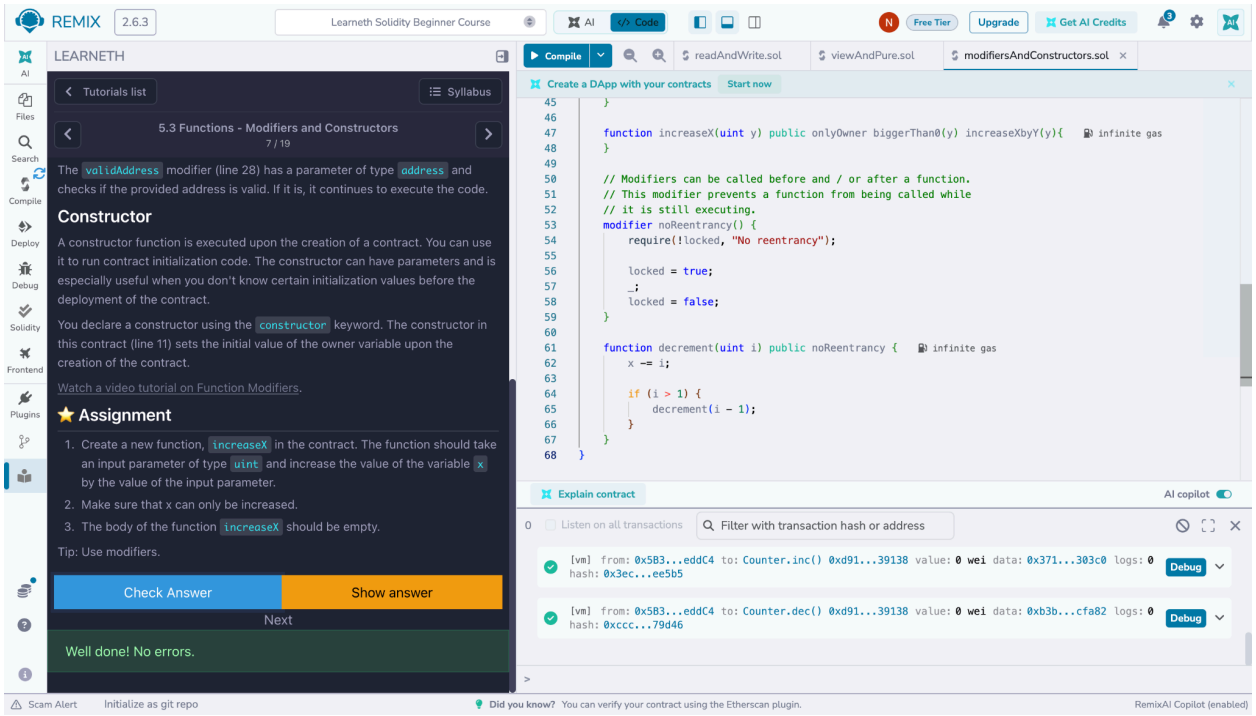
Tutorial 5 -



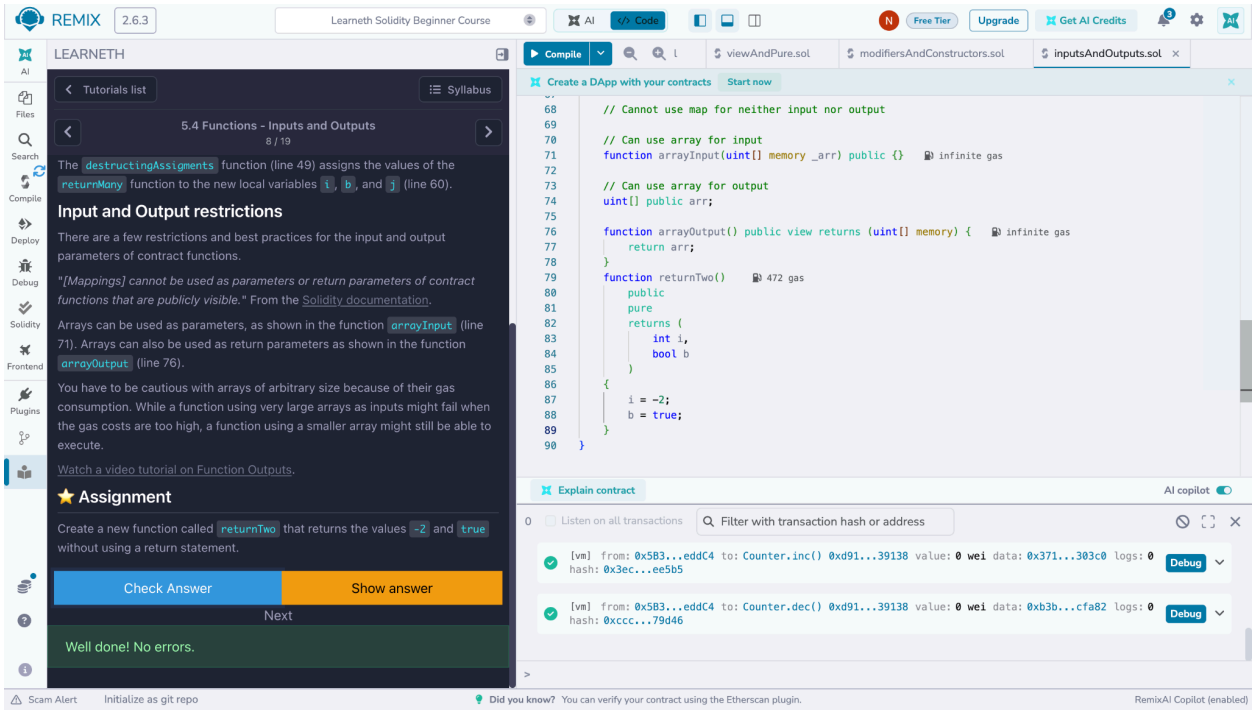
Tutorial 6 -



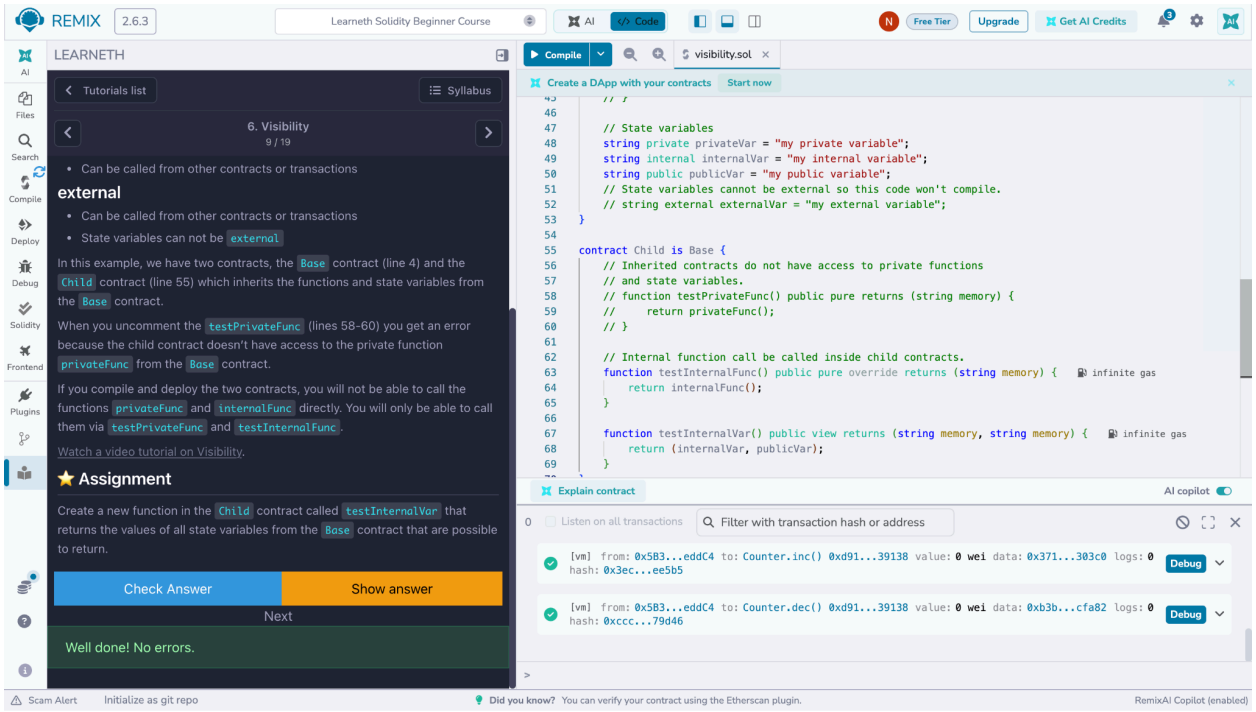
Tutorial 7 -



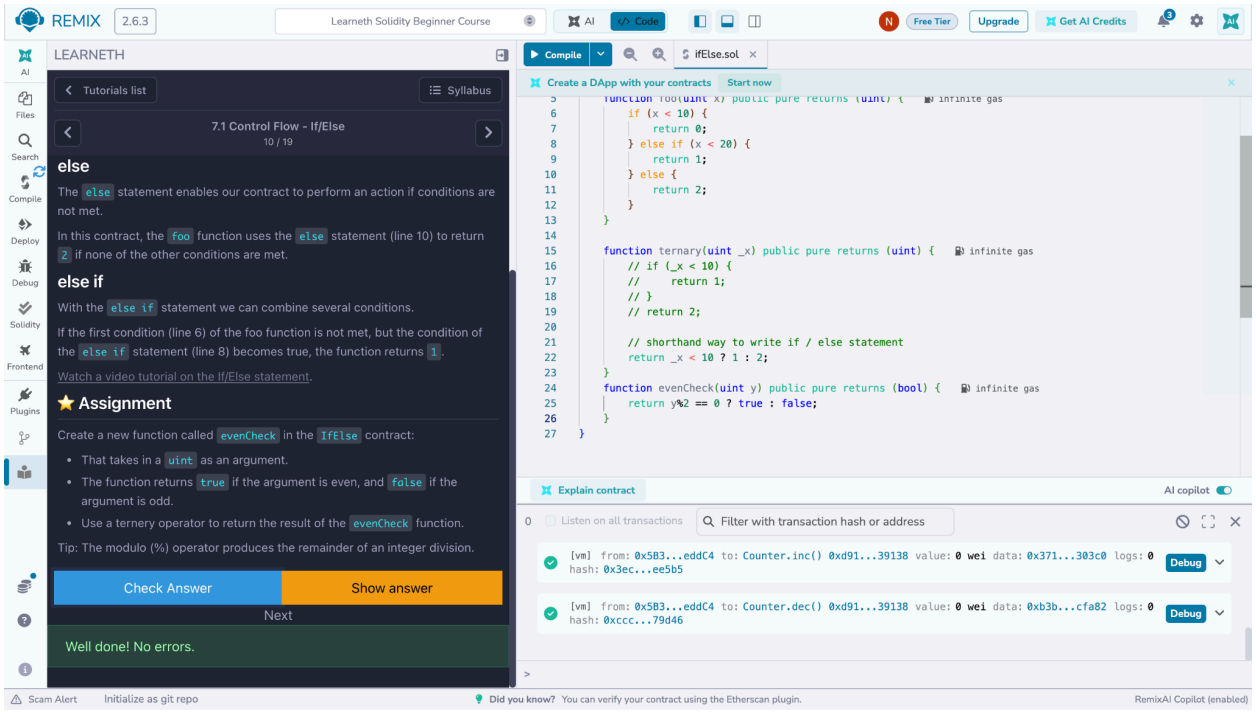
Tutorial 8 -



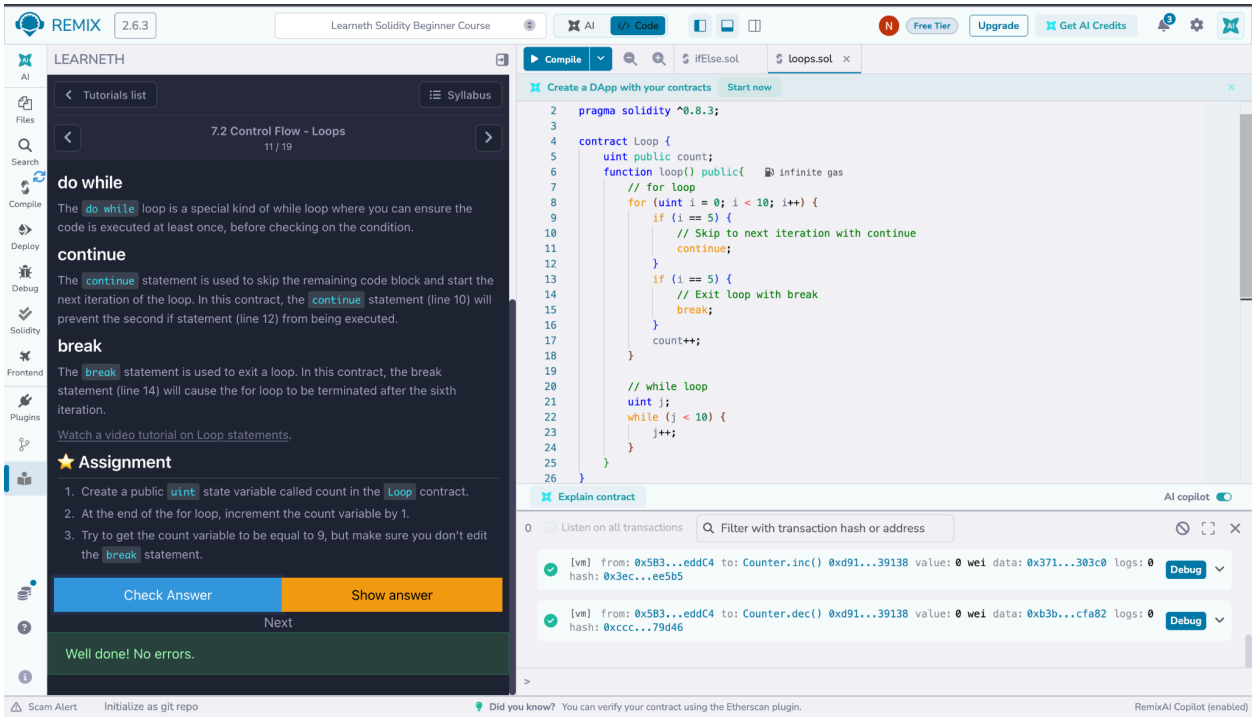
Tutorial 9 -



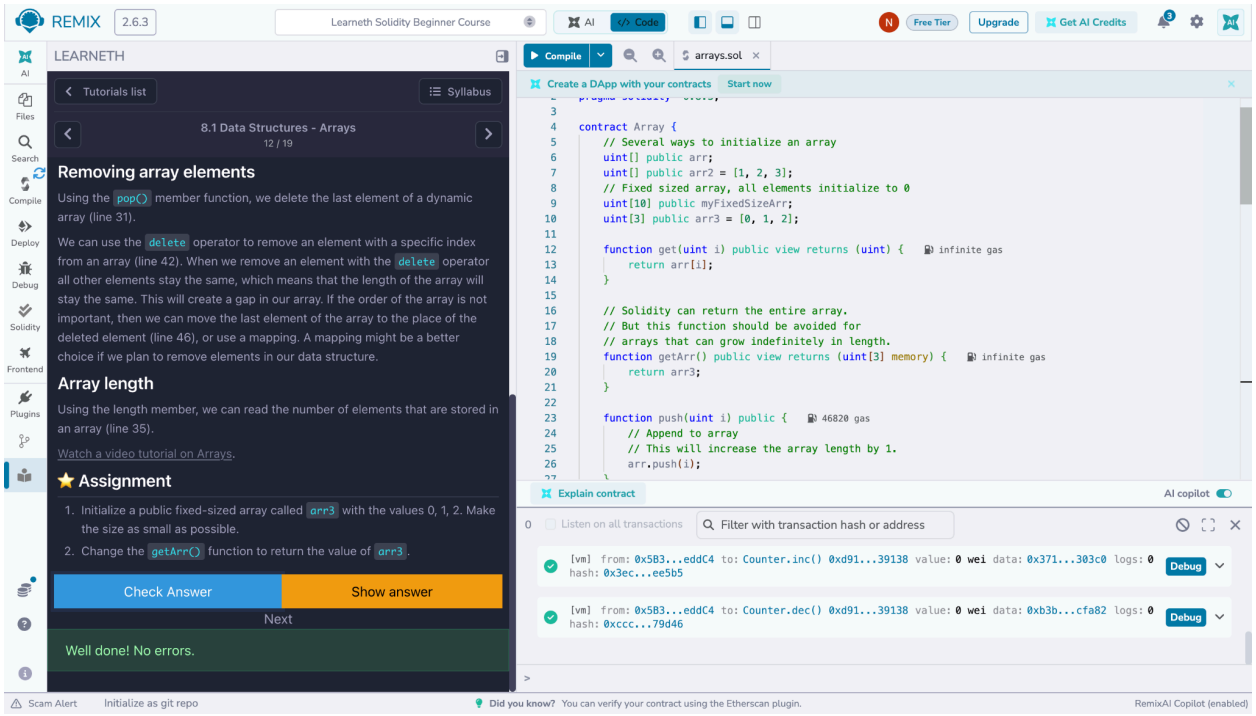
Tutorial 10 -



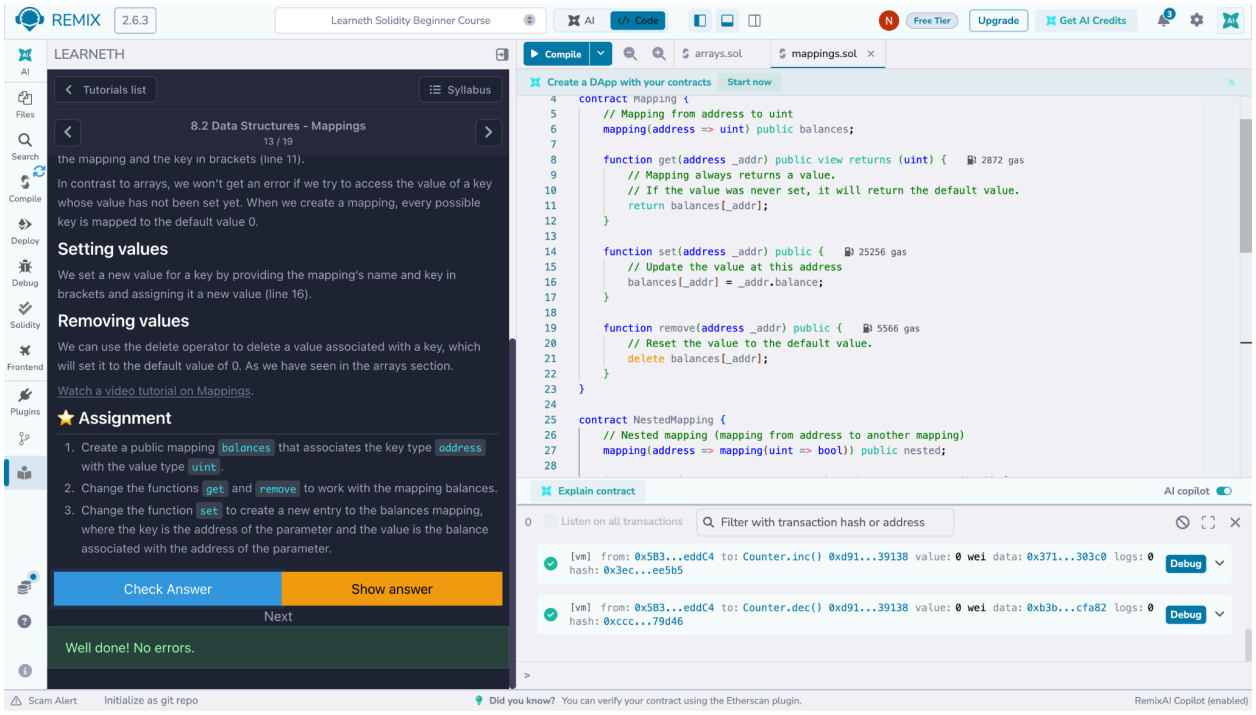
Tutorial 11 -



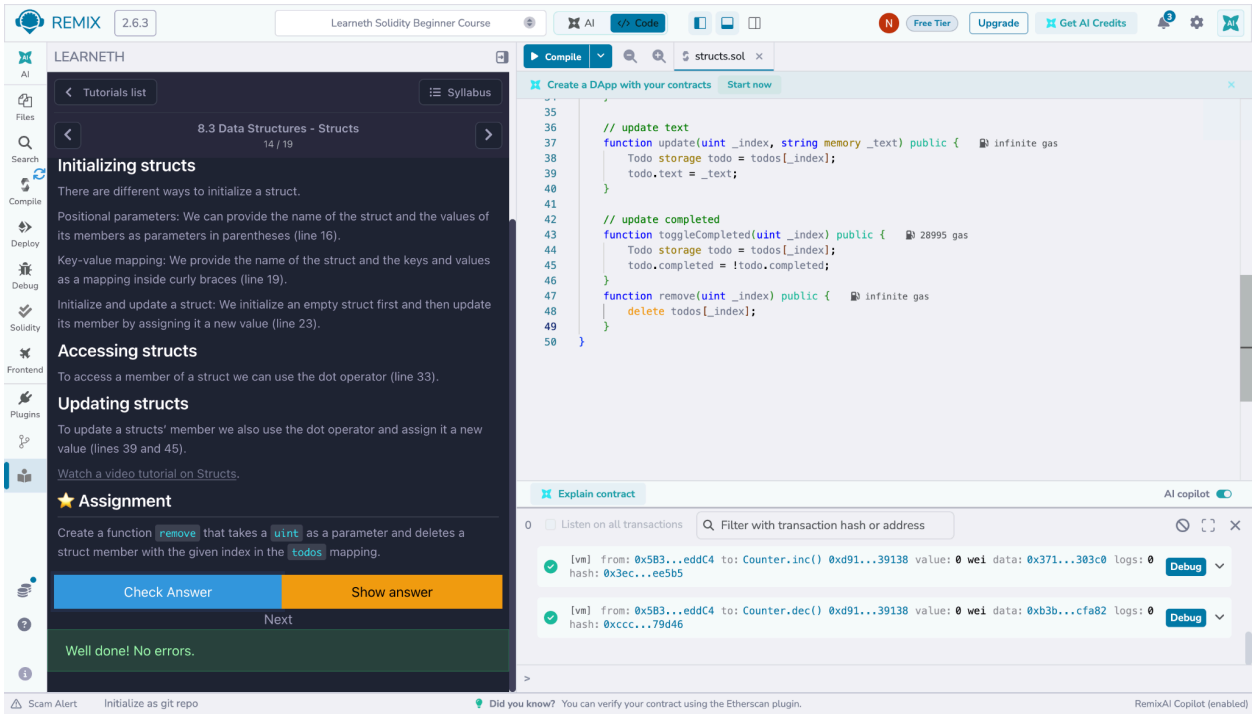
Tutorial 12 -



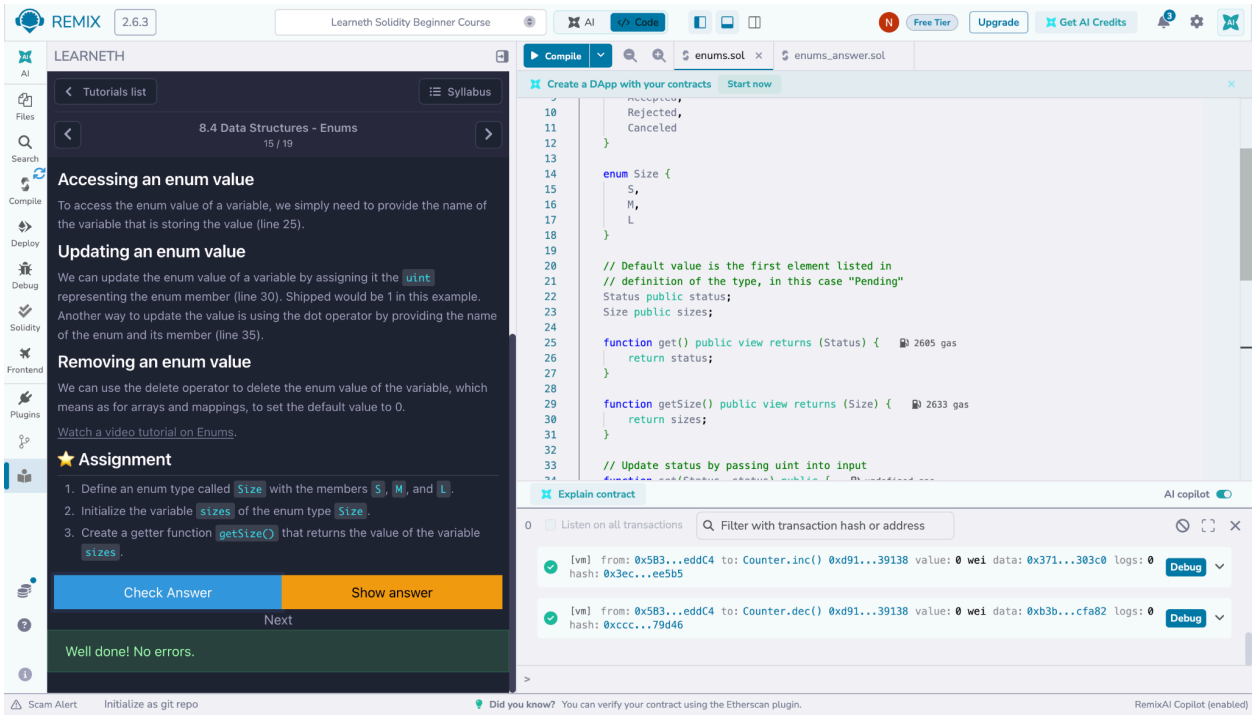
Tutorial 13 -



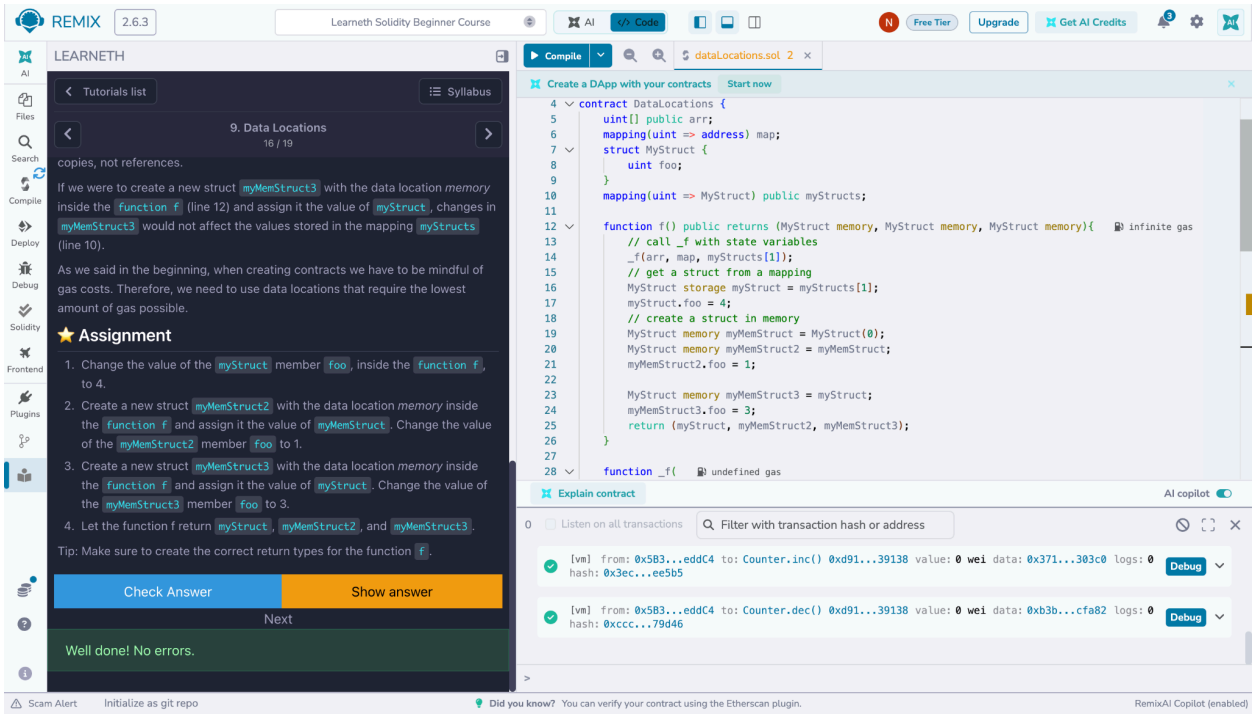
Tutorial 14 -



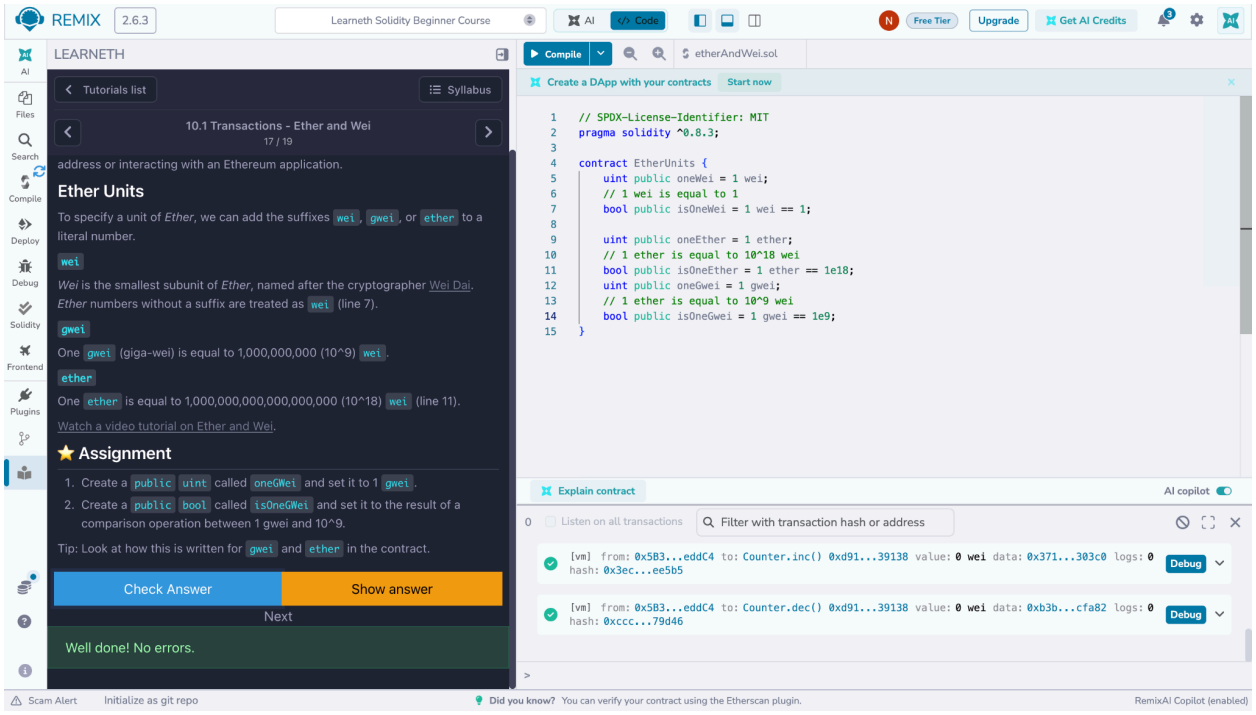
Tutorial 15 -



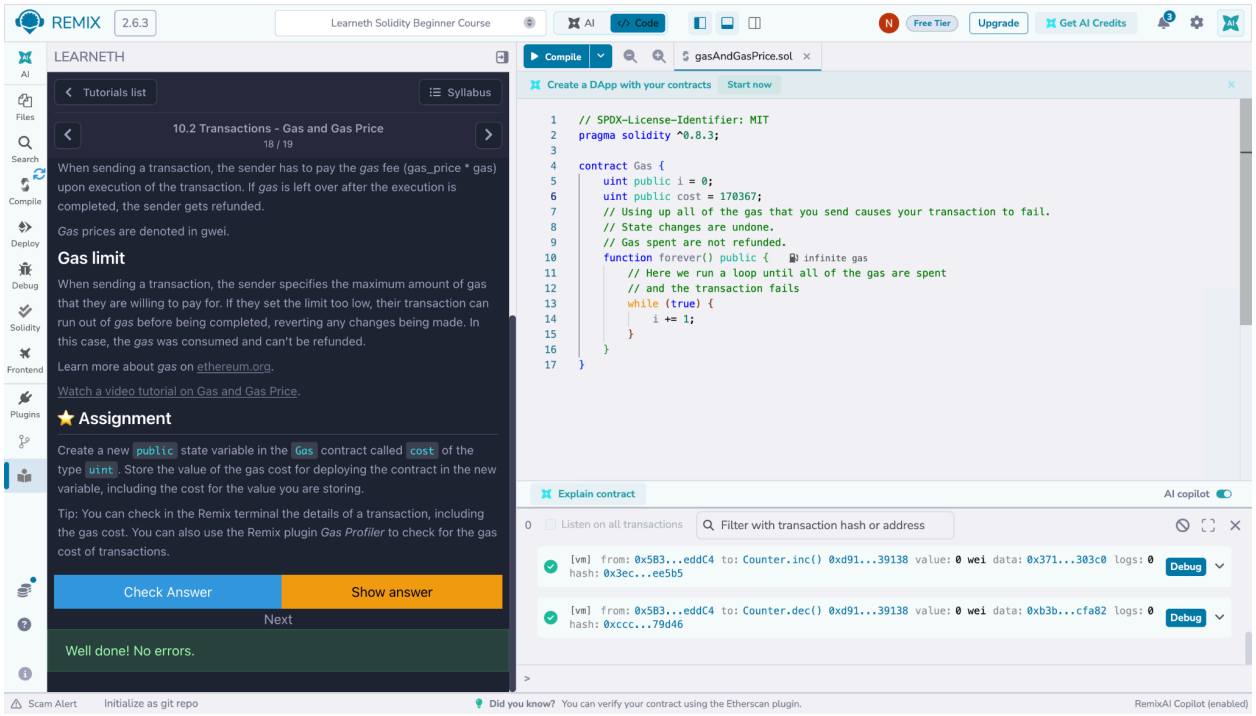
Tutorial 16 -



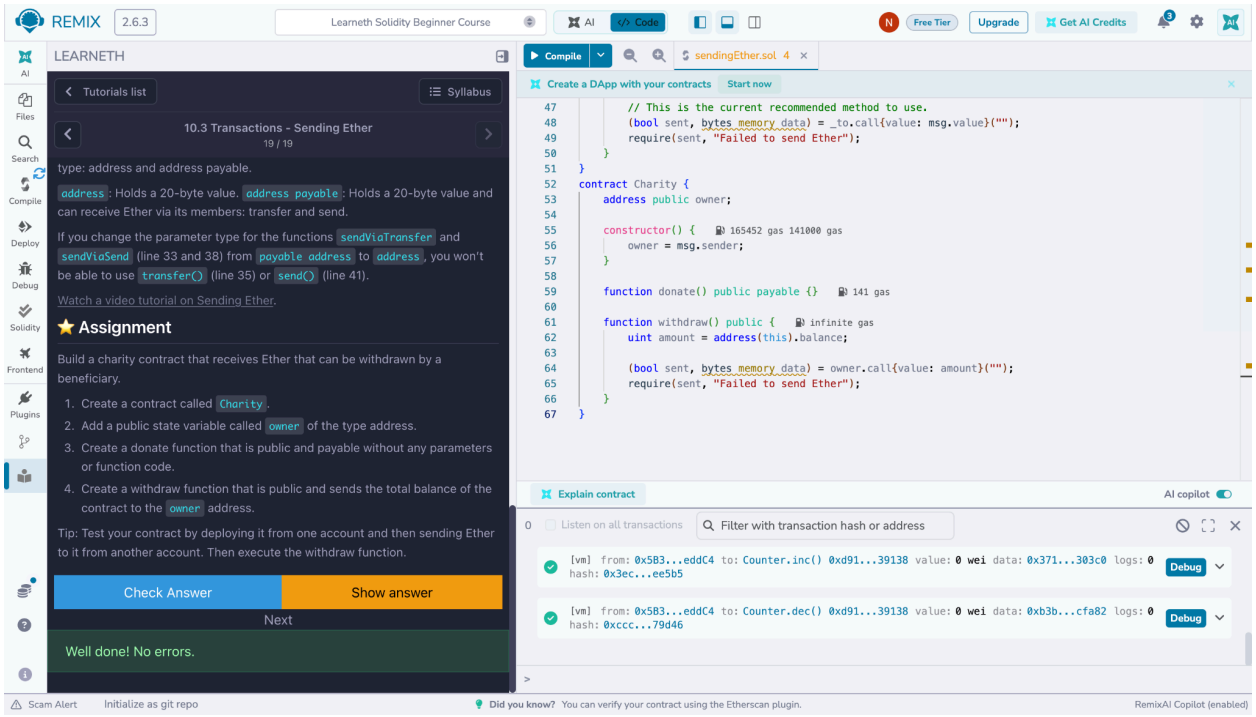
Tutorial 17 -



Tutorial 18 -


REMIX 2.6.3
Learneth Solidity Beginner Course
AI
Code
Free Tier
Upgrade
Get AI Credits
LEARNETH
Tutorials list
Syllabus
10.2 Transactions - Gas and Gas Price
18 / 19
When sending a transaction, the sender has to pay the gas fee (gas_price * gas) upon execution of the transaction. If gas is left over after the execution is completed, the sender gets refunded.
Gas prices are denoted in gwei.
Gas limit
When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.
Learn more about gas on ethereum.org.
Watch a video tutorial on Gas and Gas Price.
Assignment
Create a new public state variable in the Gas contract called cost of the type uint. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.
Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin Gas Profiler to check for the gas cost of transactions.
Check Answer
Show answer
Next
Well done! No errors.
Compile
gasAndGasPrice.sol
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Gas {
uint public i = 0;
uint public cost = 170367;
// Using up all of the gas that you send causes your transaction to fail.
// State changes are undone.
// Gas spent are not refunded.
function forever() public {
// Here we run a loop until all of the gas are spent
// and the transaction fails
while (true) {
i += 1;
}
}
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
Debug
[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
Debug
Scam Alert
Initialize as git repo
Did you know? You can verify your contract using the Etherscan plugin.
RemixAI Copilot (enabled)

Tutorial 19 -


REMIX 2.6.3
Learneth Solidity Beginner Course
AI
Code
Free Tier
Upgrade
Get AI Credits
LEARNETH
Tutorials list
Syllabus
10.3 Transactions - Sending Ether
19 / 19
type: address and address payable.
address: Holds a 20-byte value. address payable: Holds a 20-byte value and can receive Ether via its members: transfer and send.
If you change the parameter type for the functions sendViaTransfer and sendViaSend (line 33 and 38) from payable address to address, you won't be able to use transfer() (line 35) or send() (line 41).
Watch a video tutorial on Sending Ether.
Assignment
Build a charity contract that receives Ether that can be withdrawn by a beneficiary.
1. Create a contract called Charity.
2. Add a public state variable called owner of the type address.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the owner address.
Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.
Check Answer
Show answer
Next
Well done! No errors.
Compile
sendingEther.sol 4
Create a DApp with your contracts
Start now
// This is the current recommended method to use.
(bool sent, bytes memory data) = _to.call{value: msg.value}("");
require(sent, "Failed to send Ether");
}
}
contract Charity {
address public owner;
constructor() { 165452 gas 141000 gas
owner = msg.sender;
}
function donate() public payable {} 141 gas
function withdraw() public { infinite gas
uint amount = address(this).balance;
(bool sent, bytes memory data) = owner.call{value: amount}("");
require(sent, "Failed to send Ether");
}
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
Debug
[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
Debug
Scam Alert
Initialize as git repo
Did you know? You can verify your contract using the Etherscan plugin.
RemixAI Copilot (enabled)

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.